Technical Requirements Document (TRD)

AI-Powered Construction Site Intelligence Platform

Version 1.1 | Status: Engineering-Ready

1. System Overview

The AI-Powered Construction Site Intelligence Platform is a modular, computer-vision-driven safety and operations system. It ingests video from gate and zone cameras, runs YOLOv8-based detection, stores results in a relational database, and surfaces data through a FastAPI backend and React dashboard. The v1 architecture is a modular monolith with clearly separated service boundaries — designed so individual services (detection workers, API, dashboard) can be extracted into microservices later without restructuring.

The system is intentionally designed as a layered, defense-in-depth approach: the gate camera pipeline is shared across PPE verification (Module 1A) and delivery vehicle counting (Module 4A), reducing hardware cost and simplifying maintenance.

2. Recommended Tech Stack

Layer	Technology	Justification
Frontend	React.js + Tailwind CSS	Component-based architecture for role-specific views; Tailwind enables rapid UI with design-system consistency
Backend	FastAPI (Python)	Native async support, automatic OpenAPI docs, Pydantic validation out of the box
Database	PostgreSQL	Relational integrity for fraud-detection joins; FK constraints for tamper-evident audit logs; GROUP BY for dashboard KPIs
AI/CV	YOLOv8 + OpenCV + PyTorch	State-of-the-art object detection; pretrained weights available; Python-native for tight backend integration
Auth	JWT (access + refresh tokens)	Stateless, scalable, role-embeddable; industry standard for REST APIs
Containerization	Docker + Docker Compose	Environment parity across dev/staging/prod; local demo runs without cloud dependency
Cloud	AWS / Azure / Render	Flexible; Docker containers are cloud-agnostic
Version Control	Git + GitHub	Branch-based workflow with CI/CD hooks

3. Architecture Overview

The system consists of four primary layers:

- Camera Layer: Physical gate/zone cameras or pre-recorded video files (for demo). Feed is ingested by the AI Inference Layer.
- AI Inference Layer: YOLOv8 workers running PPE detection, zone monitoring, vehicle counting, and equipment classification. Workers post results to the Backend API via internal service token.
- Backend API Layer: FastAPI application handling authentication, CRUD, business logic (discrepancy calculation, hazard categorization), and serving data to the frontend.
- Presentation Layer: React dashboard displaying role-specific views — supervisor alerts, admin discrepancy escalations, material inflow logs, hazard reports.

Architecture Diagram (textual representation):

[Gate/Zone Camera] → [YOLOv8 Inference Workers] → [FastAPI Backend] → [PostgreSQL]

↓

[React Dashboard]

↓

[Alert / Notification Service]

4. Frontend Technologies

Item	Detail
Framework	React.js (functional components + hooks)
Styling	Tailwind CSS (utility-first, design-system tokens)
State Management	React Context or Zustand for global auth/role state
API Layer	Centralized in src/services/api/ — one file per resource (violations, deliveries, hazards)
Charts/KPIs	Recharts for dashboard visualizations
Routing	React Router v6 with protected route wrappers per role
Build Tool	Vite (faster dev server than CRA)

5. Backend Technologies

Item	Detail
Framework	FastAPI (Python 3.11+)
ORM	SQLAlchemy 2.x (async-compatible)
Validation	Pydantic v2 for all request/response schemas
Migrations	Alembic for schema version control
File Storage	S3-compatible object storage for violation/hazard images
Task Queue	Celery + Redis (optional, for async AI processing at scale)
AI Workers	Separate Python processes using ultralytics YOLOv8 API

6. Database Design

PostgreSQL is chosen over MongoDB because the core modules (delivery fraud detection, violations, equipment) are fundamentally relational. The discrepancy calculation requires JOINs across delivery_logs, users, and gates. COCO-based aggregate queries (compliance rate by week, material inflow by type) are clean SQL GROUP BY operations that would require verbose MongoDB aggregation pipelines.

Full schema is defined in the Database Schema Document. Key design decisions:

- All audit tables (violations, delivery_logs) are append-only — no UPDATE or DELETE endpoints exposed.
- All FKs use ON DELETE SET NULL to preserve evidence records if a related entity is removed.
- delivery_logs.discrepancy_flag is computed from count mismatch only — material_type/quantity fields are informational and do not affect fraud logic.
- material_type is constrained to an allowed list (cement, sand, bricks, steel, pipes, other) to prevent dirty data.

7. Authentication & Authorization

Item	Detail
Method	JWT — access token + refresh token pair

Item	Detail
Access Token Lifetime	15 minutes
Refresh Token Lifetime	7 days with rotation on use
Password Hashing	bcrypt (cost factor 12)
Role Storage	Role embedded in JWT payload — verified server-side on every request
User Creation	Admin-only endpoint — workers/supervisors cannot self-register
Rate Limiting	Max 5 login attempts per IP per minute — exponential backoff thereafter

8. APIs

All APIs are versioned under /api/v1/. Full endpoint specifications are in the API Documentation. Summary:

Endpoint	Method	Auth	Description
/auth/login	POST	None	Issue access + refresh tokens
/auth/refresh	POST	Refresh token	Rotate and reissue tokens
/auth/me	GET	Any authenticated	Return current user profile
/violations	GET	Admin, Supervisor	List violations with filters
/violations/{id}	GET	Admin, Supervisor	Violation detail + screenshot
/hazard-reports	POST	Worker	Submit hazard report with image
/hazard-reports	GET	Admin, Supervisor	List categorized hazard reports
/deliveries	POST	Supervisor	Log manual count + material type/quantity/unit
/deliveries/discrepancies	GET	Admin only	Escalated discrepancy list
/deliveries/materials	GET	Admin, Supervisor	Material inflow log by type over time
/dashboard/summary	GET	Admin, Supervisor	Aggregated KPIs
/equipment	GET	Admin, Supervisor	Equipment location and idle time list
/zones	POST/GET	Admin	Create and list restricted zones
/users	POST/GET	Admin	User management

9. Third-Party Integrations

Integration	Purpose	Notes
S3-compatible object storage	Store violation screenshots and hazard report images	AWS S3 / Cloudflare R2 / MinIO (self-hosted for local demo)
Email/SMS service	Deliver escalation alerts to Administrator	SendGrid / Twilio — optional for v1, can be in-app only initially
Turnstile / gate hardware	Optionally block gate on PPE failure	Site-specific integration — not in core scope, configurable

10. Security Requirements

Area	Requirement
Transport	HTTPS/TLS everywhere — no plain HTTP fallback in production
Secrets	All secrets in environment variables or secret manager — never in code or Git
Input Validation	All request bodies validated via Pydantic — unexpected fields rejected
File Uploads	Whitelist-based file type check (not blacklist); max size enforced; filename sanitized before storage
SQL Injection	SQLAlchemy ORM used everywhere — no raw string SQL
Audit Logs	violations and delivery_logs have no UPDATE/DELETE endpoints — tamper-evident
Rate Limiting	5 login attempts/min; upload endpoint rate-limited separately
CORS	Strict CORS policy — only frontend origin whitelisted
Debug Mode	Production config explicitly disables FastAPI debug mode

11. Performance Requirements

Metric	Target
Detection-to-alert latency	< 5 seconds from detection to dashboard alert
API response time (CRUD)	< 300ms for standard list/detail endpoints
Dashboard load time	< 2 seconds for summary KPI endpoint
Concurrent users	20+ simultaneous dashboard users without degradation (v1 scope)
Video inference throughput	Real-time (30fps target) on GPU inference node

12. Scalability Strategy

- Stateless FastAPI instances scale horizontally behind a load balancer — session state stored in tokens, not server memory.
- AI inference workers are GPU-bound and scale independently from the API layer — add more workers without touching the API.
- PostgreSQL read replicas for dashboard-heavy read queries at multi-site scale.
- Camera pipeline designed to accept multiple simultaneous feeds — each camera gets its own inference worker thread/process.
- Object storage (S3) scales independently of the database — images never stored in the DB itself.

13. Deployment Architecture

Local/Demo: Docker Compose — all services on one machine, no cloud dependency. Suitable for classroom demo.

Production: Containerized services behind Nginx reverse proxy with TLS termination. API instances behind load balancer. PostgreSQL on managed instance with automated backups.

Container list:

- db — PostgreSQL
- api — FastAPI application
- frontend — React build served via Nginx
- ai-worker — YOLOv8 inference worker(s)
- redis — Optional task queue broker

14. Logging & Monitoring

Area	Approach
Application logs	Structured JSON logs from FastAPI — every request logged with timestamp, user ID, route, status
AI pipeline logs	Separate log stream for inference events — model confidence, detection count, processing time per frame
Uptime monitoring	Health check endpoints (/health) polled by external monitor
AI pipeline alert	Dedicated alert if detection worker goes offline — treated as safety-critical, not generic downtime
Centralized logging	CloudWatch / ELK / Loki depending on cloud choice
Error tracking	Sentry (optional) for frontend + backend exception capture

15. Error Handling Strategy

All API errors follow a standardized response schema:

{ "error": { "code": "STRING_CODE", "message": "human readable description", "status": 400 } }

- Validation errors: 422 with field-level detail from Pydantic.
- Auth errors: 401 (missing/expired token), 403 (insufficient role).
- Not found: 404 with resource type in message.
- AI pipeline failure: log error + flag the time window as "unmonitored" in the dashboard — never silently show "no violations" when the camera is down.
- File upload errors: 400 (invalid type), 413 (too large) — never expose server path or storage details in error response.
- Discrepancy edge case: if supervisor submits manual count before AI count exists, hold comparison until AI count is logged — do not compare against null and falsely flag.